

08

HTTP & Backend İletişimi

© Gelişim · Junior → Mid-Level

🔗 Bu modülde ne öğreneceğiz?

- ✓ `provideHttpClient()` ile HTTP altyapısını kurmayı
- ✓ `HttpClient` ile veri okumayı ve `httpResource` ile ilişkisini
- ✓ Veri değiştirmeyi: `POST` / `PUT` / `DELETE`
- ✓ Sorgu parametreleri, başlıklar ve tipli cevaplar
- ✓ Functional interceptor'lar ve hata yönetimi (`retry`, `catchError`)

[8.1 Kurulum](#)
Hata[8.2 Veri Okuma](#)
Mini Proje[8.3 Veri Değiştirme](#)
Özet[8.4 Parametreler & Başlıklar](#)[8.5 Interceptor &](#)

Kurulum: provideHttpClient()

Modül 5'te `httpResource` ile veri çektin ve “bunun çalışması için `provideHttpClient()` gerekir, detayını Modül 8'de göreceğiz” demiştik. İşte o modüldeyiz: Angular'ın HTTP katmanını baştan sona kuruyoruz.

⇒ Önceki Modüllerden Hatırla

- ⇒ Modül 5: `httpResource` ile veri çekmek ve `isLoading` / `error` / `hasValue` kalıbı.
- ⇒ Modül 7: Veriyi `@Service()` içine taşımak (`ProductService`).
- ⇒ Bu modül ikisini birleştirir: HTTP çağrıları **servis katmanında** yaşar.

Nedir?

`HttpClient`, Angular'ın sunucuyla konuşma aracıdır. Tarayıcının ham `fetch` 'i yerine onu kullanırsınız; çünkü Angular'ın DI'ına, interceptor'larına ve test altyapısına bağlıdır.

Neden ayrıca “provide” etmek gerekiyor?

Angular, uygulamanın yalnızca **ihtiyaç duyduğu** parçaları yüklesin ister. HTTP kullanmayan bir uygulama bu kodu taşımasın diye, HTTP katmanı varsayılan olarak açık değildir: `app.config.ts` içinde açıkça sağlarsın.

TYPESCRIPT

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    // ...diğer provider'lar (provideRouter vb.)
  ],
};
```

Bu tek satır, uygulamanın her yerinde `HttpClient` ve `httpResource` kullanılabilir hale getirir.

Ek özellikler (feature'lar)

`provideHttpClient` 'a **özellik fonksiyonları** vererek davranışını ayarlarsın. En sık kullanılan `withInterceptors` 'dır (8.5'te göreceğiz):

TYPESCRIPT

```
provideHttpClient(  
  withInterceptors([loggingInterceptor, authInterceptor]),  
)
```

Varsayılan olarak `fetch`

`HttpClient` istekleri varsayılan olarak tarayıcının `fetch` API'si üzerinden yapar; bu, sunucu tarafı render (SSR) kullanan uygulamalar için performans ve uyum açısından önerilen yoldur. Yani ayrıca bir şey yapmana gerek yok.

🔔 Mini Quiz

1. `provideHttpClient()` nereye yazılır ve ne işe yarar?
↳ `app.config.ts` içindeki `providers` dizisine; uygulama genelinde `HttpClient/httpResource`'u kullanılabilir yapar.
2. Neden HTTP katmanı varsayılan olarak açık değil?
↳ Kullanmayan uygulamalar gereksiz kod taşımaz diye (isteğe bağlı, opt-in).
3. `withInterceptors` ne için kullanılır?
↳ HTTP isteklerine araya giren (middleware) fonksiyonlar eklemek için.

✓ Bu Dersten Çıkarılacaklar

- ✓ `HttpClient` = Angular'ın sunucuyla konuşma aracı (DI + interceptor + test uyumlu).
- ✓ `provideHttpClient()` `app.config.ts` 'e eklenir; olmadan HTTP çalışmaz.
- ✓ Özellikler fonksiyonlarla eklenir: `withInterceptors([...])`.
- ✓ İstekler varsayılan olarak `fetch` üzerinden gider.

Ders 8.2

<> Veri Okuma: `httpResource` ve `HttpClient.get`

Veri okumanın iki yolu var ve **ikisi de doğru** — hangisini seçeceğin, veriyi nasıl kullanacağına bağlı.

Yol 1: `httpResource` — ekrana bağlı veri

Modül 5'te öğrendiğin yol. Veriyi **signal** olarak verir; URL'deki signal değişince kendini yeniler. Ekranda gösterilecek veriler için idealdir. Şimdi bunu, Modül 7'de yazdığın servisin içine taşıyalım:

TYPESCRIPT

```
import { Service, signal, computed } from '@angular/core';  
import { httpResource } from '@angular/common/http';
```

```
import { Product } from '../models/product';

@Service()
export class ProductService {
  // Modül 7'deki sabit dizi yerine: canlı veri
  private readonly response = httpResource<ProductsResponse>(
    () => 'https://dummyjson.com/products',
  );

  // Component'lerin kullanacağı sade yüzey
  readonly products = computed(() => this.response.value()?.products ?? []);
  readonly isLoading = this.response.isLoading;
  readonly error = this.response.error;

  reload(): void {
    this.response.reload();
  }
}
```

🔗 Neden servise taşıyoruz?

Modül 7'nin dersi: veri erişimi component'in değil **servisin** işidir. Böylece birden çok component aynı veriye tek noktadan ulaşır; component sadece `productService.products()` der, URL'i bile bilmez.

Yol 2: `HttpClient.get` — tek seferlik çağrı

Bazen veriyi ekrana bağlamazsın; bir kez alıp bir işlem yaparsın (bir hesap, bir doğrulama, bir dışa aktarma). O zaman doğrudan `HttpClient` kullanırsın:

TYPESCRIPT

```
import { Service, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Service()
export class ReportService {
  private readonly http = inject(HttpClient);

  downloadSummary(): void {
    this.http
      .get<ProductsResponse>('https://dummyjson.com/products')
      .subscribe((response) => {
        console.log('Toplam ürün:', response.total);
      });
  }
}
```

```
});  
  
}  
  
}
```

⚠ **subscribe** olmadan istek gitmez

HttpClient metotları bir **Observable** döndürür ve bu “tembeldir”: **subscribe** etmedikçe istek **hiç gönderilmez**. Bu, yeni başlayanların en sık takıldığı noktadır. (**httpResource** kullanırken böyle bir dert yok; o kendisi yönetir.)

Hangisini ne zaman?

Durum	Seçim
Veri ekranda gösterilecek, durumu (loading/error) lazım	httpResource
Parametre değişince otomatik yenilensin	httpResource
Tek seferlik okuma, sonucu ekrana bağlı değil	HttpClient.get + subscribe
Veri değiştirme (POST/PUT/DELETE)	HttpClient (8.3)

🔗 Mini Quiz

- httpResource** ile **HttpClient.get** arasındaki temel fark nedir?
↳ **httpResource** veriyi signal olarak verir ve durumu yönetir; **HttpClient.get** tek seferlik Observable döndürür.
- Neden HTTP çağrılarını servise taşırız?
↳ Veri erişimi servisin işidir; birçok component tek noktadan aynı veriye ulaşır (Modül 7).
- HttpClient.get** çağırdın ama istek gitmiyor — neden?
↳ **subscribe** etmediğin için — Observable tembeldir.

✗ Sık Yapılan Hatalar

- ✗ **subscribe** unutmak. İstek hiç gitmez.
- ✗ HTTP çağrısını component'e yazmak. Servise taşı (Modül 7).
- ✗ Sarmalayıcı tipi atlamak. DummyJSON `{ products: [...] }` döndürür; tipini doğru kur (Modül 5).

✓ Bu Dersten Çıkarılacaklar

- ✓ **httpResource** : ekrana bağlı veri; signal + otomatik durum yönetimi.
- ✓ **HttpClient.get** : tek seferlik okuma; **subscribe** şart.
- ✓ HTTP çağrıları servis katmanında yaşar; component sade yüzeyi kullanır.

✓ Servis `computed` ile temiz bir arayüz sunar (`products()` , `isLoading()`).

Ders 8.3

⇒ Veri Değiştirme: POST, PUT, DELETE

Şimdiye kadar hep **okuduk**. Gerçek uygulamalar veri de **yazar**: yeni kayıt ekler, günceller, siler. Burada `httpResource` kullanılmaz — çünkü o okuma içindir. Yazma işlemleri doğrudan `HttpClient` ile yapılır.

Üç temel metot

Metot	Ne yapar	Kullanım
<code>post()</code>	Yeni kayıt oluşturur	<code>http.post<T>(url, body)</code>
<code>put()</code>	Var olan kaydı günceller	<code>http.put<T>(url, body)</code>
<code>delete()</code>	Kayıdı siler	<code>http.delete<T>(url)</code>

TYPESCRIPT

```
import { Service, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Service()
export class ProductService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = 'https://dummyjson.com/products';

  addProduct(title: string, price: number): void {
    this.http
      .post<Product>(`${this.baseUrl}/add`, { title, price })
      .subscribe((created) => {
        console.log('Oluşturuldu:', created.id);
      });
  }

  updateProduct(id: number, price: number): void {
    this.http.put<Product>(`${this.baseUrl}/${id}`, { price }).subscribe();
  }

  deleteProduct(id: number): void {
    this.http.delete<Product>(`${this.baseUrl}/${id}`).subscribe();
  }
}
```

```
}  
  
}
```

İkinci parametre (`body`) sunucuya gönderdiğin veridir; Angular onu otomatik olarak JSON'a çevirir ve uygun başlığı ekler. Dönüş tipini `<Product>` ile belirtirsin, böylece cevap tipli gelir.

Yazdıktan sonra listeyi tazelemek

Klasik senaryo: yeni ürün ekledin, listenin güncellenmesi lazım. İki yol var:

1) Yeniden çek (en güvenli): yazma başarılı olunca `reload()` çağırıp sunucudan taze veriyi al.

TYPESCRIPT

```
addProduct(title: string, price: number): void {  
  this.http.post<Product>(`${this.baseUrl}/add`, { title, price }).subscribe(() => {  
    this.response.reload(); // httpResource'u tazele (Modül 5)  
  });  
}
```

2) Yerel olarak güncelle (daha hızlı hissettirir): sunucudan dönen kaydı yerel signal'e ekle. Modül 5'teki immutability kuralı burada da geçerlidir:

TYPESCRIPT

```
private readonly _localProducts = signal<Product[]>([]);  
  
addProduct(title: string, price: number): void {  
  this.http.post<Product>(`${this.baseUrl}/add`, { title, price }).subscribe((created) => {  
    this._localProducts.update((list) => [...list, created]); // yeni dizi!  
  });  
}
```

⇒ Önceki Modüllerden Hatırla

- ⇒ Modül 5: Immutable güncelleme — `[...list, item]`, `push` değil.
- ⇒ Modül 5: `reload()` ile `httpResource` 'u elle tazeleme.
- ⇒ Modül 7: Durumu `private signal` + `asReadonly()` ile korumak.

📄 DummyJSON yazma işlemleri simüledir

Alıştırma yaparken: DummyJSON `POST` / `PUT` / `DELETE` isteklerini kabul eder ve gerçekmiş gibi cevap döner, ama **kalıcı olmaz** (sunucuda bir şey değişmez). Öğrenmek için yeterlidir; “ekledim ama listede yok” diye şaşırma.

🔔 Mini Quiz

1. Yeni kayıt oluşturmak için hangi metod ve parametreler?

↳ `http.post<T>(url, body)` — ikinci parametre gönderilen veridir.

2. Neden yazma işlemlerinde `httpResource` kullanılmaz?

↳ `httpResource` okuma (read) içindir; yazma işlemleri doğrudan `HttpClient` ile yapılır.

3. Ekleme sonrası listeyi güncelleme için iki yolu nedir?

↳ `reload()` ile yeniden çekmek, ya da dönen kaydı yerel signal'e immutable şekilde eklemek.

✗ Sık Yapılan Hatalar

✗ Yazma işlemini `httpResource` ile yapmaya çalışmak. O okuma içindir.

✗ `subscribe` unutmak. POST/PUT/DELETE de gönderilmez.

✗ Yerel listeyi mutasyonla güncellemek. Yeni referans ver (Modül 5).

✓ Bu Dersten Çıkarılacaklar

✓ `post` / `put` / `delete` ile veri yazılır; hepsi `subscribe` ister.

✓ Gövde (body) ikinci parametredir; Angular JSON'a çevirir.

✓ Yazma sonrası: `reload()` ya da yerel immutable güncelleme.

✓ `httpResource` okuma, `HttpClient` yazma işleri içindir.

Ders 8.4

🛡️ Sorgu Parametreleri, Başlıklar & Tipli Cevaplar

Gerçek API'lerle çalışırken URL'e parametre eklemen, başlık göndermen ve cevabı doğru tiplenmen gerekir. Üçü de kısa ama kritik konular.

Sorgu parametreleri: `params`

URL'e elle `?limit=10&skip=20` yazmak yerine `params` kullanırsın; Angular kodlamayı (encoding) doğru yapar:

TYPESCRIPT

```
this.http.get<ProductsResponse>('https://dummyjson.com/products', {
  params: { limit: 10, skip: 20 },
}).subscribe();

// Arama: /products/search?q=phone
this.http.get<ProductsResponse>('https://dummyjson.com/products/search', {
```



```
params: { q: 'phone' },
}).subscribe();
```

Aynısını `httpResource` ile yaparken URL'i signal'lerden kurarsın — parametre değişince istek kendiliğinden yenilenir (Modül 5):

TYPESCRIPT

```
private readonly query = signal('');

private readonly searchResponse = httpResource<ProductsResponse>(
  () => `https://dummyjson.com/products/search?q=${this.query()}`,
);
```

Başlıklar: `headers`

Kimlik doğrulama jetonu, dil tercihi gibi bilgiler başlıkla gönderilir:

TYPESCRIPT

```
this.http.get<Product>(url, {
  headers: { 'Accept-Language': 'tr-TR' },
}).subscribe();
```

🔔 Her istekte tekrar etme

Aynı başlığı (örneğin `Authorization`) her çağrıda elle eklemek hem tekrardır hem unutmaya açıktır. Doğru yol: bir **interceptor** yazıp başlığı merkezi olarak eklemek — sıradaki derste.

Tipli cevaplar

Angular cevabı otomatik JSON olarak çözer; sen sadece **ne beklediğini** söylersin. Bu, Modül 2'deki interface bilgisinin doğrudan karşılığıdır:

TYPESCRIPT

```
export interface ProductsResponse {
  products: Product[];
  total: number;
  skip: number;
  limit: number;
}

// Beklenen tip generic ile verilir
```

```
this.http.get<ProductsResponse>(url);  
httpResource<ProductsResponse>(() => url);
```

⇒ Önceki Modüllerden Hatırla

- ⇒ Modül 2: `interface` ile veri şeklini tanımlamak.
- ⇒ Modül 5: Sarmalayıcı cevap (`{ products, total }`) tipini doğru kurmak.
- ⇒ Modül 6: Gelen ham veriyi ekranda pipe'larla biçimlendirmek (`currency` , `date`).

⚠ Tip bir söz dür, doğrulama değil

`get<Product>` demek, Angular'ın cevabı doğruladığı anlamına gelmez; sadece TypeScript'e “bunu bekliyorum” dersin. API farklı bir şey dönerse derleyici uyarır, hata çalışma anında ortaya çıkar. Bu yüzden API dokümanını dikkatle oku.

🔔 Mini Quiz

1. URL'e parametre eklemenin doğru yolu nedir?
↳ `params` seçeneği ile (Angular kodlamayı doğru yapar).
2. Aynı başlığı her istekte göndermek için en iyi yol?
↳ `Interceptor` yazmak (8.5) — merkezî ve unutulmaz.
3. `get<Product>` cevabı doğrular mı?
↳ Hayır; yalnızca TypeScript'e beklenen tipi bildirir, çalışma anında doğrulama yapmaz.

✓ Bu Dersten Çıkarılacaklar

- ✓ `params` ile sorgu parametreleri; `httpResource` 'ta URL signal'lerden kurulur.
- ✓ `headers` ile başlık; tekrar edenler `interceptor`'a taşınır.
- ✓ Cevap tipi generic ile bildirilir (`get<T>`); bu bir söz, doğrulama değil.
- ✓ Sarmalayıcı cevaplarda tipi doğru kurmak kritik (Modül 5 dersi).

Ders 8.5

🚀 Interceptor'lar & Hata Yönetimi

Her isteğe jeton eklemek, her hatayı loglamak, hepsini tek yerden yönetmek... **Interceptor** tam bunun içindir.

Interceptor nedir?

Interceptor, giden her isteğin ve gelen her cevabın arasından geçtiği bir fonksiyondur — bir ara katman (middleware). İstekleri değiştirebilir, cevapları izleyebilir, hataları yakalayabilir.

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';

// Her isteğe kimlik jetonu ekler
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token();

  // İstekler değiştirilemez; clone ile kopyasını üretirsin
  const authorized = req.clone({
    setHeaders: { Authorization: `Bearer ${token}` },
  });

  return next(authorized);
};
```

Kaydetmek için `provideHttpClient` 'a verirsin. Sıra önemlidir: listede önce yazılan isteği önce işler.

```
provideHttpClient(
  withInterceptors([loggingInterceptor, authInterceptor]),
),
```

📌 Fonksiyonel interceptor'lar tercih edilir

Interceptor'ların sınıf tabanlı bir yazımı da vardır, ancak Angular **fonksiyonel interceptor'ları** (`withInterceptors`) önerir: sıralamaları daha öngörülebilirdir ve `inject()` ile doğrudan servis alabilirler.

Hata yönetimi

HTTP hataları `HttpErrorResponse` olarak gelir. Bir isteği yeniden denemek ya da hatayı dönüştürmek için RxJS operatörleri kullanılır:

```
import { catchError, retry, throwError } from 'rxjs';

this.http.get<ProductsResponse>(url)
  .pipe(
    retry(2), // başarısız olursa 2 kez daha dene
    catchError((error: HttpErrorResponse) => {
```

```
// 404 mü, 500 mü, ağ hatası mı?  
console.error('İstek başarısız:', error.status);  
return throwError(() => new Error('Ürünler yüklenemedi'));  
}),  
)  
.subscribe();
```

Aynı işi merkezî olarak da yapabilirsin — bir hata interceptor'ı tüm isteklerin hatalarını tek yerden yakalar:

TYPESCRIPT

```
export const errorInterceptor: HttpInterceptorFn = (req, next) =>  
  next(req).pipe(  
    catchError((error: HttpErrorResponse) => {  
      inject(NotificationService).showError(error.status);  
      return throwError(() => error);  
    })  
  );
```

❓ **httpResource** kullanıyorsan hata zaten signal

httpResource ile çekiyorsan hatayı **error()** signal'inden okur, şablonda **@else if (error())** ile gösterirsin (Modül 5). Interceptor ise **çapraz kesen** işler içindir: loglama, bildirim, jeton yenileme.

❓ **Mini Quiz**

1. Interceptor ne yapar ve nasıl kaydedilir?

↳ Giden istek/gelen cevabı araya girip işler; **provideHttpClient(withInterceptors([...]))** ile kaydedilir.

2. İstek nesnesini neden doğrudan değiştiremezsin?

↳ İstekler değişmezdir (immutable); **req.clone({...})** ile kopyasını üretirsin.

3. Bir isteği başarısız olduğunda tekrar denemek için?

↳ **retry(n)** operatörü ile (RxJS **pipe** içinde).

✗ **Sık Yapılan Hatalar**

- ✗ **req** 'i doğrudan değiştirmeye çalışmak. **req.clone()** kullan.
- ✗ Her serviste aynı başlığı elle eklemek. Interceptor'a taşı.
- ✗ **next(req)** 'i döndürmeyi unutmak. Zincir kırılır, istek gitmez.

✓ Bu Dersten Çıkarılacaklar

- ✓ Interceptor = her istek/cevabın geçtiği ara katman (fonksiyonel yazım önerilir).
- ✓ `req.clone({ setHeaders })` ile istek kopyalanıp değiştirilir; `next(req)` döndürülür.
- ✓ Hatalar `HttpErrorResponse`; `retry` ve `catchError` ile yönetilir.
- ✓ `httpResource` kullanıyorsan hata `error()` signal'indedir; interceptor çapraz işler için.

Bölüm

🚩 Mini Proje: Ürün Yönetim Paneli (CRUD)

Bu modülün kavramlarını, önceki modüllerde öğrendiklerinle birleştiren kapsamlı bir alıştırma: ürünleri **listele**yen, **ekle**yen ve **sil**en bir yönetim paneli.

Ürün Yönetimi

DummyJSON API · listele, ekle, sil

Ekle

30 ürünün 4 tanesi

Essence Mascara Lash Princess	\$9.99	99 adet	×
Eyeshadow Palette with Mirror	\$19.99	14 adet	×
Powder Canister	\$14.99	Tükendi	×
Red Lipstick	\$12.99	68 adet	×

Ürün yönetim paneli: ekleme formu, arama, durum rozetleri ve silme

Özellikler / Gereksinimler

- **Listeleme:** ürünler `httpResource` ile **servis içinden** gelsin; yükleniyor/hata durumları olsun.
- **Ekleme:** form doldurulup gönderilince `POST` isteği atılsın, ardından liste tazelensin.
- **Silme:** satırdaki × ile `DELETE` isteği atılsın, liste güncellensin.
- **Arama:** gelen listeyi `computed` ile yerel süz (Modül 5).
- **Biçimlendirme:** fiyat `currency`, stok durumu rozeti (Modül 6).
- **Stok rozeti:** kendi directive'inle renklensin — stok 0 ise kırmızı, azsa turuncu, bolsa yeşil (Modül 6).

Angular tarafı (sende) — ipuçları

- **Servis (Modül 7 + 8):** `@Service()` içinde `httpResource` ile liste + `inject(HttpClient)` ile `post` / `delete`.

- **Sade yüzey:** servis dışarı `products` (computed), `isLoading`, `error` ve `addProduct` / `deleteProduct` versin.
- **Tazeleme:** yazma işlemi `subscribe` içinde bittiğinde `reload()` çağır.
- **Form:** alanları signal'de tut, `[value]` + `(input)` ile bağla (Modül 4/5).
- **Directive (Modül 6):** `[appStockLevel]="product.stock"` — selector adıyla aynı input, `host` + `computed` ile renk.
- **Boş durum:** `@empty` ile “ürün yok” mesajı.

⇒ Bu Projede Birleşen Modüller

- ⇒ Modül 5: `httpResource`, `computed` ile yerel arama, immutable güncelleme.
- ⇒ Modül 6: `currency` pipe, custom directive (`host` + selector adlı input).
- ⇒ Modül 7: `@Service()` + `inject()`, servis dışına sade/okunur yüzey.
- ⇒ Modül 8: `provideHttpClient`, `post` / `delete`, `subscribe`, `reload()`.

① Materyaller

Statik referansı ayrıca verdim (JS yok): `index.html`, `styles.scss`, `styles.css` ve ekran görüntüsü. API olarak DummyJSON kullan (`/products`, `/products/add`, `/products/{id}`). Yazma işlemleri simüledir; bu yüzden ekleme sonrası listede görünmeyebilir — isteğin gittiğini DevTools'un Network sekmesinden doğrula (Modül 3).

✓ Bu Dersten Çıkarılacaklar

- ✓ Tam CRUD akışı: okuma `httpResource`, yazma `HttpClient`.
- ✓ Servis katmanı: veri erişimi + sade yüzey (products/isLoading/error + metotlar).
- ✓ Önceki modüller birlikte: signal/computed, pipe, custom directive, DI.
- ✓ Yazma sonrası tazeleme (`reload()`) alışkanlığı.

Bölüm

🚩 Modül 8 Özeti

Artık uygulaman dış dünyayla tam anlamıyla konuşabiliyor: okuyor, yazıyor, hatayı yönetiyor.

- **Kurulum:** `provideHttpClient()` (+ `withInterceptors`).
- **Okuma:** ekrana bağlıysa `httpResource`, tek seferlikse `HttpClient.get`.
- **Yazma:** `post` / `put` / `delete` + `subscribe`; sonra `reload()`.
- **Ayrıntılar:** `params`, `headers`, generic ile tipli cevaplar.
- **Çapraz işler:** interceptor'lar; hatalarda `retry` / `catchError`.

① Sırada ne var?

Modül 9 — Routing & Navigation: rotaların tam anlatımı — child routes, lazy loading, route guard'ları, query parametreleri ve gezinme desenleri. Modül 5'te “minimal” kullandığımız routing'ın bütün hâli.

Takıldığın her yeri sor — mini projeyi yapıp gösterdiğinde birlikte gözden geçiririz. Kolay gelsin!